

Summary of Inference: Beyond Text

Vaelone Elankumaran (vaelone), Himanish Kolli (himak), Ayan Nair (ayannair), Mohammed Almakrami (almkrami)

Lecture Date: October 5th, 2026

1. Research direction of field

Multimodal generative models expand inference beyond text to images, audio, and video, introducing heterogeneous computation paths and resource demands. Traditional systems often treat an entire model or workflow as a single serving unit, but modern pipelines combine components such as encoders, language models, diffusion models, and decoders with very different compute and memory requirements. Scaling these components together can waste GPU resources and limit sharing across applications.

Recent work shifts toward exposing and managing these components independently. *DiFlow* decomposes text-to-image diffusion pipelines into independently scalable model-execution nodes, enabling model sharing and adaptive parallelism. *Cornfigurator* extends this idea to Any-to-Any multimodal models by automatically selecting among colocated, disaggregated, and hybrid deployments based on the workload. Systems such as Cornserve further support component disaggregation, independent scaling, and efficient tensor transfer. Together, these papers suggest that multimodal inference is evolving from optimizing monolithic models toward managing distributed computation graphs, where systems must decide how heterogeneous components should be handled across GPU clusters.

2. Lecture papers

Paper 1: *Cornfigurator: Automated Planning for Any-to-Any Multimodal Model Serving*

What is the problem of this paper?

The problem is that Any-to-Any (A2A) multimodal models are difficult to deploy efficiently because different types of requests can take different computation paths and place different demands on resources. It is thus hard/illogical to use a single deployment for all request types

What is the solution of this paper?

The paper proposes Cornfigurator, an automated deployment planner that finds an efficient deployment strategy for generic A2A models. Instead of treating all requests as one workload, Cornfigurator builds plans around individual request types, identified by their input and output modalities. It creates subplans for individual request types, combines them into logical plans, and then translates them into physical resource allocations. Configurator progressively evaluates and prunes these plans using Monte Carlo, Pareto analysis, and a detailed request-level simulator.

How is the solution evaluated?

They chose to evaluate based on goodput—the number of requests that complete within their latency targets. Cornfigurator is evaluated on two AWS p4de.24xlarge instances, each containing eight NVIDIA A100-80GB GPUs, using several A2A models and workloads. The authors compare its resulting deployments against both generic A2A deployment strategies and specialized strategies, finding $1.24\text{--}2.73\times$ higher goodput than generic approaches and $1.12\text{--}6.32\times$ higher goodput than specialized approaches. They demonstrate that the planner can reduce roughly 483 million plans to 5 candidates, selecting a final plan in about 116 seconds.

Overall, what are 1-2 shortcomings of this paper?

One shortcoming is the use of the Monte Carlo estimates to prune plans, which are unable to capture dependencies and wait for requests. 99.6% of plans are pruned in this stage, so it is

possible optimal ones are lost because of this gap. Another potential shortcoming is the paper's definition of goodput. Since latency targets are based on computational requirements, multimodal requests receive lenient targets. This could result in high goodput even if multimodal requests have poor absolute latency, despite being a core workload of the system.

Paper 2: DiFlow: A System for Micro-Serving Text-to-Image Diffusion Workflows

What is the problem of this paper?

The problem is that diffusion model workflows are difficult to deploy efficiently because they contain multiple models that can have different resource requirements and execution patterns. Existing systems generally treat the entire workflow as one unit, meaning models must be scaled and scheduled together. This wastes GPU resources, prevents different workflows from sharing loaded models, and limits parallelism/ flexible resource allocation.

What is the solution of this paper?

The paper proposes a micro-serving system that treats each model in a diffusion workflow independently. Users define workflows through a Python DSL, which DiFlow compiles into a DAG. DiFlow then uses fine-grained scheduling to enable model sharing across workflows and adaptive parallelism.. It also uses NVSHMEM for efficient GPU-to-GPU tensor transfers and admission control to reject requests that are unlikely to meet their latency SLO.

How is the solution evaluated?

DiFlow is evaluated on 12 workflows using SD3, SD3.5-Large, Flux-Schnell, and Flux-Dev, with experiments using 8–32 H800 GPUs and a 256-GPU simulator for larger-scale tests. The authors compare DiFlow against Diffusers. The main metric is SLO attainment (the percentage of submitted requests that finish within their latency deadline). DiFlow sustains up to $3\times$ higher request rates at 90% SLO attainment, performs well under stricter latency requirements, handles $8\times$ greater traffic burstiness, and achieves 90% SLO attainment with up to $3\times$ fewer GPUs

Overall, what are 1-2 shortcomings of this paper?

One potential shortcoming is in the evaluation. Only 4 models are used which can make it hard to have an overall/general conclusion. Something else to keep in mind is the scheduling. Within the paper it is stated that scheduling was purposefully kept simple, with adjusted FCFS. Using different/optimized scheduling patterns may result in microservices having different effects.

3. Lecture discussions

1. For Cornfigurator, why was the latency target set at P25 instead of others like P50 or P90? P25 was chosen arbitrarily, as it is not the most relaxed nor the tightest bound without industry standard. A sweep should have been conducted to determine the bound.

2. For DiFlow, at what point do communication overheads outweigh the benefits of disaggregation? Since there was a high level of model sharing and local stores/zero-copy sharing implemented, the overhead is small in comparison to total time for the workloads.

3. For Cornfigurator, how does the improved goodput outweigh planning work and model migration? The models are fixed, so migration is not a main concern; any changes to the model is treated as a new model, causing replanning and migration regardless. Furthermore, all profiling has already been done for new requests that use existing components.

4. Could models benefit from profiling on different types of GPUs instead of using one type of GPU? Yes, in theory models could be profiled on different GPUs and thus could be used for different request types, but this would also enlarge the search space.

5. For Cornfigurator, would high-volume request types be biased? Yes, optimizations will create starvation gaps in terms of goodput vs. fairness, but this is minimized with the steady state assumption. The long-term average ratio matters most despite short-term variation.

6. Will the planner ever become too complicated? Yes. The discussion that followed related the planner's search space to that of database query plans. The current solution for this problem would be pruning, rules, and heuristics to find an approach, since finding the optimum in an infinitely large search space would be unrealistic.

7. For DiFlow, how can we tell if improvement is from parallelism or from the workflow decomposition? They would be difficult to differentiate in terms of improvement, but it was mentioned that they are related, as disaggregation allows for parallelism to be implemented.

4. Proposal for follow-up work

DiFlow considers model-loading costs but leaves long-term replica placement open (Sections 5.1 and 8), while Cornfigurator supports workload-based replanning (Section 4.4). Building on Section 1's focus on independently managed components, we ask when placement changes justify their temporary disruption. Our focus is deciding when to switch, rather than treating loading-cost awareness itself as novel.

We would build a controller for one diffusion backbone and two ControlNet variants, switching only when predicted deadline completions exceed those from staying in place, including transition losses over a fixed interval. DiFlow would retain batching, admission control, and dispatch within placement constraints. Affected executors would stop accepting tasks, finish running work, and preserve or transfer needed tensors before unloading. Memory checks and staggered loads would limit disruption.

We would first verify that different request mixes favor different placements. A simulator calibrated with GPU measurements and selected hardware runs would compare unmodified DiFlow, fixed placement, and immediate, persistence-based, and proposed switching policies using identical placement options and demand estimates. Stable, bursty, and shifting workloads would test deadline attainment, counting rejections as failures, per-workflow p95 latency, reload frequency, and transition performance. Gains beyond persistence-based switching would support the approach; similar placement performance, forecast errors, or interference erasing gains would challenge it.

5. A reflection on AI-Usage

Vaelone: For section 2, I used AI to collect my thoughts. I took notes and used AI to turn those into responses. I fine tuned those to remove redundancy, and added in some key points it missed.

Ayan: For section 1, I mainly used AI to give me an understanding of the content of the two papers as well as connecting both of them to the research topic. I then asked it to use its previous response as primary context for creating a draft of the research direction. Then I refined the draft response by myself. One thing I noticed was that the first response had an extremely long response, and I would have preferred if the response was more compacted.

Mohammed: I used AI to connect the two papers and draft a follow-up proposal. It helped organize the idea, but needed revisions to distinguish our contribution from DiFlow and clarify the controller and evaluation. I also refined the wording and shortened the proposal while keeping its main points.